



Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Технології розроблення програмного забезпечення
Лабораторна робота №4
на тему «ШАБЛони «SINGLETON», «ITERATOR», «PROXY»,
«STATE», «STRATEGY»»

Виконала:
Студентка групи ІА-22
Патрик Вікторія

Перевірив:
Мякий Михайло Юрійович

Київ 2024

Зміст

Тема	3
Короткі теоретичні відомості.....	3
Реалізація не менше 3-х класів відповідно до обраної теми	5
Реалізація шаблону за обраною темою	6
1. SyntaxHighlightStrategy	6
2. JavaSyntaxHighlightStrategy	7
3. HtmlSyntaxHighlightStrategy	7
4. PlainTextHighlightStrategy	8
5. SyntaxHighlightContext	8
6. SyntaxHighlightController.....	9
Діаграма класів для патерну «Strategy»	10
Переваги застосування патерну «Strategy»	10

ШАБЛОНИ «SINGLETON», «ITERATOR», «PROXY», «STATE», «STRATEGY»

Мета: ознайомитися з шаблонами проектування «Singleton», «Iterator», «Proxy», «State», «Strategy» та набути практичних навичок їх застосування. Реалізувати частину функціоналу програми за допомогою одного з розглянутих шаблонів для досягнення конкретних функціональних можливостей та забезпечення ефективної взаємодії між класами.

Тема:

..3 Текстовий редактор (strategy, command, observer, template method, flyweight, SOA)

Текстовий редактор повинен вміти розпізнавати текстові файли в будь-якій кодуванні, мати розширені функції редагування: макроси, сніппети, підказки, закладки, перехід на рядок / сторінку, підсвічування синтаксису (для однієї мови програмування або розмітки на розсуд студента).

Хід роботи:

Короткі теоретичні відомості:

Шаблони проектування – це формалізовані рішення типових завдань, які часто виникають при розробці інформаційних систем. Вони містять опис завдання, вдале рішення та рекомендації щодо його використання в різних ситуаціях. Кожен шаблон має загальноприйняту назву, що дозволяє легко ідентифікувати та повторно використовувати його.

Важливим етапом роботи з шаблонами є правильне моделювання предметної області, що допомагає формалізувати задачу і вибрати відповідний шаблон. Використання шаблонів проектування надає розробнику низку переваг: вони роблять систему структурованішою, простішою у вивченні та розширенні, підвищують її стійкість до змін і полегшують інтеграцію з іншими системами.

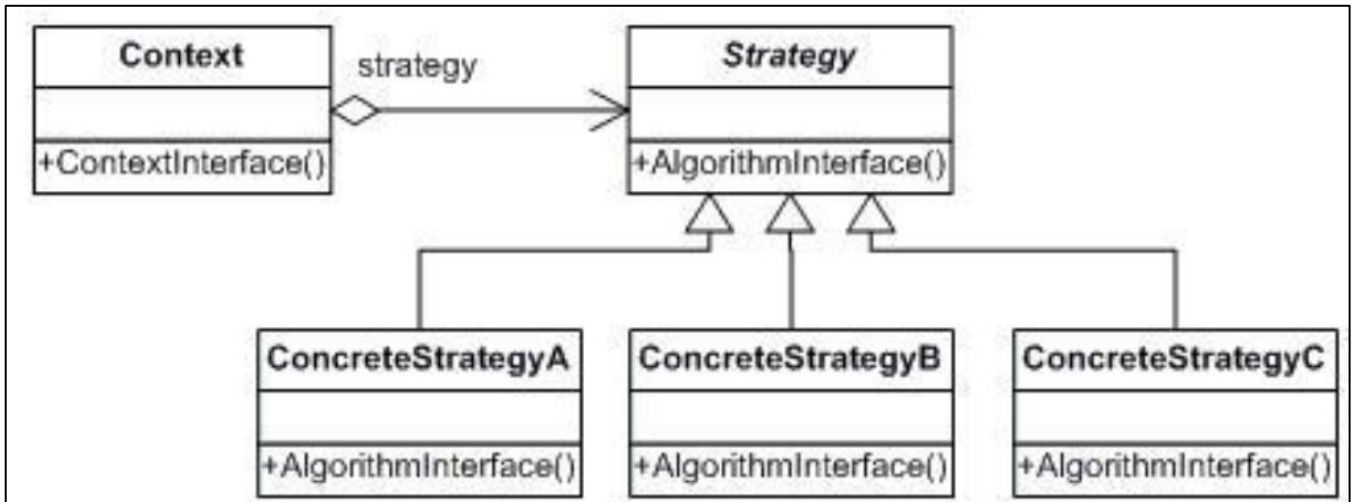
Таким чином, шаблони є перевіреними рішеннями, які використовуються в різних компаніях і проектах для створення архітектурних рішень у відповідних обставинах.

Навіщо використовувати шаблони?

Шаблони проектування зменшують час і трудовитрати на створення архітектури, надають системі гнучкість та полегшують її підтримку. Вони полегшують спільне розуміння архітектури в команді та є економічно виправданими, але їх варто використовувати обдуманно, а не в кожному проекті.

Шаблон «Strategy»

Структура:



Призначення:

Паттерн "Стратегія" дозволяє легко змінювати поведінку об'єкта, використовуючи різні алгоритми, які досягають тієї ж мети. Наприклад, це може бути реалізація різних підходів до сортування або обробки даних. Основна ідея полягає у тому, щоб кожен алгоритм мав окремий клас, що реалізує спільний інтерфейс, і їх можна було взаємозамінювати.

Проблема:

При створенні програми часто виникає потреба реалізувати кілька різних варіантів алгоритмів для однієї задачі. Наприклад, навігатор для прокладання маршрутів може використовувати автомобільні, пішохідні, велосипедні маршрути або навіть оптимальні шляхи громадським транспортом. Реалізація всіх алгоритмів в одному класі призводить до його розростання і складності, ускладнюючи підтримку та модифікацію коду. Зміна одного алгоритму може випадково зачепити інші, а додавання нового варіанту потребує переписування існуючого коду.

Рішення:

Паттерн "Стратегія" пропонує винести кожен алгоритм у власний клас. Контекст взаємодіє зі стратегіями через спільний інтерфейс, тому залишається незалежним від їхньої реалізації. Щоб змінити алгоритм, достатньо передати новий об'єкт-стратегію в контекст. Це значно спрощує підтримку і розширення коду, адже додавання нової стратегії не вимагає змін у самому контексті.

Приклад із життя:

Потрібно дістатися до аеропорту. Ви можете вибрати між таксі, автобусом чи велосипедом залежно від бюджету та часу. У цьому випадку вид транспорту є стратегією, а ви обираєте відповідну стратегію відповідно до своїх потреб.

Переваги:

- Легке переключення між алгоритмами.
- Ізоляція логіки алгоритмів у різних класах.
- Принцип відкритості/закритості: можна додавати нові стратегії без змін у контексті.

Недоліки:

- Збільшується кількість класів у проєкті.
- Клієнт повинен розуміти, яку стратегію вибрати для конкретного випадку.

Цей підхід ідеально підходить для задач, де часто змінюються чи додаються нові варіанти поведінки.

Реалізація не менше 3-х класів відповідно до обраної теми:

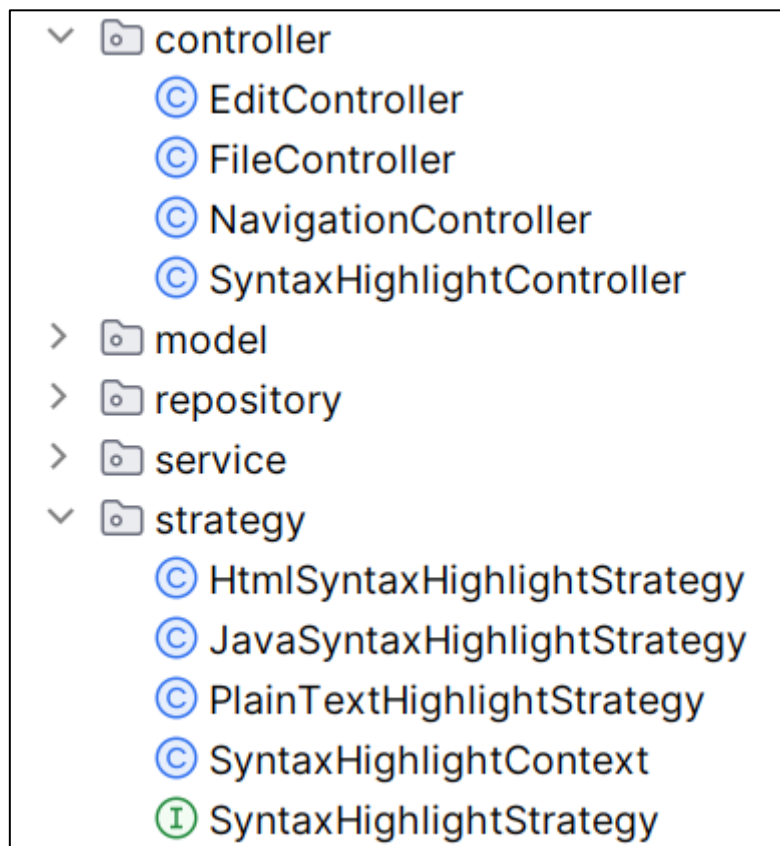


Рисунок 1. Структура проєкту з використанням патерну «Стратегія».

У ході лабораторної роботи була реалізована система текстового редактора, основним завданням якої було використання патерну **Стратегія** для реалізації підсвічування синтаксису тексту. Її можна побачити на Рисунку 1. В результаті виконання роботи були виконані наступні кроки:

1. **Розроблено загальну структуру проєкту:** проєкт поділено на логічні пакети, такі як controller, model, repository, service, та strategy, що забезпечує зручність розширення та підтримки коду.
2. **Реалізовано патерн "Стратегія":**
 - Створено загальний інтерфейс SyntaxHighlightStrategy, який визначає поведінку підсвічування синтаксису.
 - Розроблено три окремі стратегії:
 - JavaSyntaxHighlightStrategy — підсвічування синтаксису Java.
 - HtmlSyntaxHighlightStrategy — підсвічування синтаксису HTML.
 - PlainTextHighlightStrategy — залишення тексту без змін.
 - Контекст SyntaxHighlightContext забезпечує динамічну зміну стратегій під час виконання програми.
3. **Створено контролери для роботи з функціоналом:**
 - SyntaxHighlightController обробляє запити користувача, вибирає відповідну стратегію та повертає підсвічений текст.
4. **Перевірено роботу системи:**
 - Через тестування API були отримані результати, які демонструють правильну роботу кожної стратегії для обробки тексту.

Даний підхід забезпечує гнучкість системи, дозволяючи легко додавати нові види підсвічування без змін у вже існуючому коді.

Реалізація шаблону за обраною темою:

1. SyntaxHighlightStrategy



```
1 package com.example.text_editor.strategy;
2
3 public interface SyntaxHighlightStrategy {
4     String highlight(String text);
5 }
```

Рисунок 2. Інтерфейс SyntaxHighlightStrategy.

Цей інтерфейс, який ми бачимо на Рисунок 2 визначає спільний метод `highlight(String text)`, який реалізується в кожній стратегії підсвічування синтаксису. Використовується для забезпечення єдиного підходу до обробки тексту незалежно від конкретної стратегії.

2. JavaSyntaxHighlightStrategy

```
1 package com.example.text_editor.strategy;
2
3 import org.springframework.stereotype.Component;
4
5 @Component
6 public class JavaSyntaxHighlightStrategy implements SyntaxHighlightStrategy {
7
8     1 usage
9     @Override
10     public String highlight(String text) {
11         return text.replaceAll( regex: "\\b(public|class|void|static)\\b", replacement: "[HIGHLIGHT]$1[/HIGHLIGHT]");
12     }
13 }
```

Рисунок 3. Клас JavaSyntaxHighlightStrategy.

Цей клас, який ми бачимо на Рисунку 3 реалізує інтерфейс `SyntaxHighlightStrategy` та відповідає за підсвічування синтаксису мови Java. Реалізація використовує регулярні вирази для знаходження ключових слів Java (наприклад, `public`, `class`, `static`, `void`) і додає теги для їх виділення.

3. HtmlSyntaxHighlightStrategy

```
1 package com.example.text_editor.strategy;
2
3 import org.springframework.stereotype.Component;
4
5 @Component
6 public class HtmlSyntaxHighlightStrategy implements SyntaxHighlightStrategy {
7
8     1 usage
9     @Override
10     public String highlight(String text) {
11         return text.replaceAll( regex: "<([\\w+])>", replacement: "[TAG]<$1[/TAG]");
12     }
13 }
```

Рисунок 4. Клас HtmlSyntaxHighlightStrategy.

Цей клас, який ми бачимо на Рисунку 4 реалізує інтерфейс `SyntaxHighlightStrategy` та забезпечує підсвічування HTML-коду. Він знаходить HTML-теги у тексті (наприклад, `<html>`, `<body>`) і виділяє їх спеціальними тегами для підсвічування.

4. PlainTextHighlightStrategy

```
1 package com.example.text_editor.strategy;
2
3 import org.springframework.stereotype.Component;
4
5 @Component
6 public class PlainTextHighlightStrategy implements SyntaxHighlightStrategy {
7
8     1 usage
9     @Override
10    public String highlight(String text) {
11        return text;
12    }
13 }
```

Рисунок 5. Клас PlainTextHighlightStrategy.

Цей клас, який ми бачимо на Рисунку 5 реалізує інтерфейс SyntaxHighlightStrategy, але не змінює текст. Він використовується для випадків, коли підсвічування не потрібне, і текст залишається без змін.

5. SyntaxHighlightContext

```
1 package com.example.text_editor.strategy;
2
3 import org.springframework.stereotype.Service;
4
5 2 usages
6 @Service
7 public class SyntaxHighlightContext {
8
9     3 usages
10    private SyntaxHighlightStrategy strategy;
11
12    3 usages
13    public void setStrategy(SyntaxHighlightStrategy strategy) { this.strategy = strategy; }
14
15    1 usage
16    public String highlight(String text) {
17        if (strategy == null) {
18            throw new IllegalStateException("Strategy is not set");
19        }
20        return strategy.highlight(text);
21    }
22 }
```

Рисунок 6. Клас SyntaxHighlightContext.

Цей клас, який ми бачимо на Рисунку 6 є контекстом для патерну "Стратегія". Він містить посилання на поточну стратегію підсвічування (SyntaxHighlightStrategy) і дозволяє динамічно змінювати її через метод

setStrategy(SyntaxHighlightStrategy strategy). Метод highlight(String text) викликає реалізацію підсвічування, відповідну обраній стратегії.

6. SyntaxHighlightController

```
1 package com.example.text_editor.controller;
2
3 import com.example.text_editor.strategy.*;
4 import org.springframework.web.bind.annotation.*;
5
6 @RestController
7 @RequestMapping("/syntax")
8 public class SyntaxHighlightController {
9
10     5 usages
11     private final SyntaxHighlightContext context;
12     2 usages
13     private final JavaSyntaxHighlightStrategy javaStrategy;
14     2 usages
15     private final HtmlSyntaxHighlightStrategy htmlStrategy;
16     2 usages
17     private final PlainTextHighlightStrategy plainStrategy;
18
19     public SyntaxHighlightController(SyntaxHighlightContext context,
20                                     JavaSyntaxHighlightStrategy javaStrategy,
21                                     HtmlSyntaxHighlightStrategy htmlStrategy,
22                                     PlainTextHighlightStrategy plainStrategy) {
23
24         this.context = context;
25         this.javaStrategy = javaStrategy;
26         this.htmlStrategy = htmlStrategy;
27         this.plainStrategy = plainStrategy;
28     }
29 }
```

Рисунок 7.1. Контролер SyntaxHighlightController.

```
24
25 @PostMapping("/highlight")
26 public String highlight(@RequestParam String text, @RequestParam String type) {
27     switch (type.toLowerCase()) {
28         case "java":
29             context.setStrategy(javaStrategy);
30             break;
31         case "html":
32             context.setStrategy(htmlStrategy);
33             break;
34         default:
35             context.setStrategy(plainStrategy);
36     }
37     return context.highlight(text);
38 }
39 }
```

Рисунок 7.2. Контролер SyntaxHighlightController.

Цей контролер, який ми бачимо на Рисунках 7.1 та 7.2 відповідає за обробку запитів на підсвічування синтаксису. Він отримує текст (text) і тип підсвічування (type) через HTTP-запит, вибирає відповідну стратегію (наприклад, Java, HTML, Plain Text) і повертає підсвічений текст. Контролер використовує SyntaxHighlightContext для роботи зі стратегіями.

Діаграма класів для патерну «Strategy»:

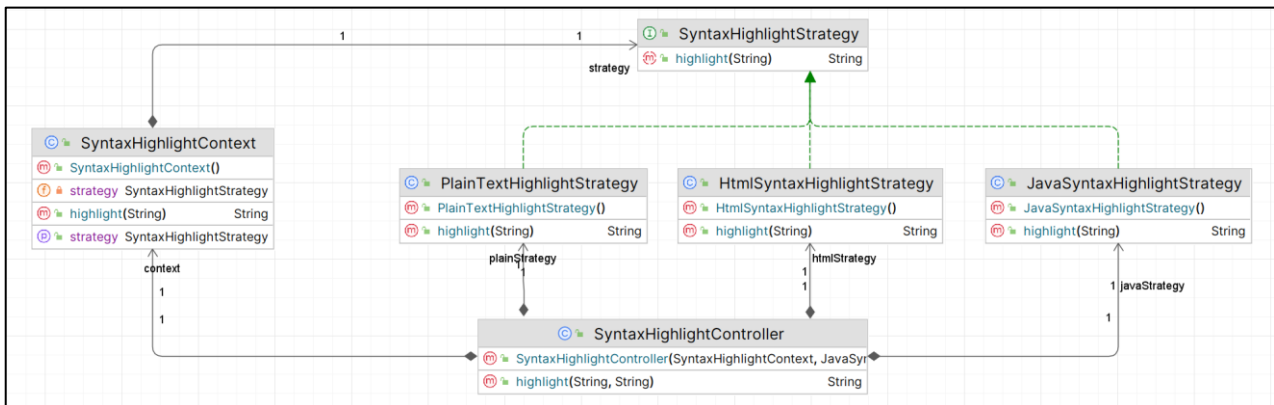


Рисунок 8. Діаграма класів для патерну «Strategy».

Раніше всі алгоритми підсвічування могли б бути реалізовані в одному класі, що призвело б до:

- Ускладнення підтримки коду через додавання нових типів підсвічування.
- Ризику внесення помилок при модифікації алгоритмів.
- Порушення принципу відкритості/закритості, адже будь-яка зміна потребувала б редагування основного класу.

Патерн «Strategy» вирішив цю проблему, дозволивши:

- Розділити алгоритми підсвічування на окремі класи.
- Динамічно змінювати поведінку програми залежно від обраного типу тексту.

Переваги застосування патерну «Strategy»:

1. Гнучкість:

- Легко додавати нові стратегії підсвічування (наприклад, для інших мов програмування) без змін у контексті або контролері.

2. Принцип відкритості/закритості:

- Існуючі класи не потрібно змінювати для додавання нової функціональності.

3. Читаємість і підтримка:

- Логіка кожного алгоритму ізольована в окремому класі, що спрощує тестування і розуміння коду.

4. Модульність:

- Кожен алгоритм працює незалежно, що дозволяє легко замінити або оновити окрему стратегію без впливу на решту системи.

Висновок: завдяки даній лабораторній роботі я реалізувала патерн проєктування «Strategy» для текстового редактора, що дозволило створити гнучкий механізм підсвічування синтаксису для різних типів тексту (Java, HTML, звичайний текст). Завдяки цьому патерну, алгоритми підсвічування були винесені в окремі класи, що спростило підтримку та розширення функціональності.

Переваги застосування патерну «Strategy» включають ізоляцію логіки кожного алгоритму, гнучкість у додаванні нових типів підсвічування та зменшення залежності між компонентами. Реалізація цього патерну зробила систему більш модульною, зрозумілою та легкою для масштабування в майбутньому.